

```

import os
import torch
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from PIL import Image
import matplotlib.pyplot as plt
from google.colab import drive
drive.mount('/content/drive')

```

Mounted at /content/drive

```
dataset_path = "/content/drive/MyDrive/ANNlab/Pistachio_Image_Dataset/Pistachio_Image_Dataset"
```

```

transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.125])
])

```

```

class PistachioDataset(Dataset):
    def __init__(self, root_dir, transform=None):
        self.root_dir = root_dir
        self.transform = transform
        self.images = []
        self.labels = []

        # Load Class 0: Kirmizi_Pistachio
        kirmizi_folder = os.path.join(root_dir, "Kirmizi_Pistachio")
        for file in os.listdir(kirmizi_folder):
            if file.lower().endswith(('jpg', 'jpeg', 'png')):
                self.images.append(os.path.join(kirmizi_folder, file))
                self.labels.append(0)

        # Load Class 1: Siirt_Pistachio
        siirt_folder = os.path.join(root_dir, "Siirt_Pistachio")
        for file in os.listdir(siirt_folder):
            if file.lower().endswith(('jpg', 'jpeg', 'png')):
                self.images.append(os.path.join(siirt_folder, file))
                self.labels.append(1)

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        img_path = self.images[idx]
        label = self.labels[idx]
        image = Image.open(img_path).convert("L")

        if self.transform:
            image = self.transform(image)

        label = torch.tensor(label, dtype=torch.long)
        return image, label

```

```

dataset = PistachioDataset(dataset_path, transform=transform)
dataloader = DataLoader(dataset, batch_size=64, shuffle=True)

```

```

class_name = ['Kirmizi_Pistachio', 'Siirt_Pistachio']
image, labels = next(iter(dataloader))

```

```

plt.figure(figsize=(10, 10))
for i in range(10):
    plt.subplot(5, 5, i + 1)
    img = image[i].permute(1, 2, 0)
    img = img * 0.125 + 0.5 # denormalize
    plt.imshow(img.numpy(), cmap='gray')
    plt.title(f"{class_name[labels[i].item()]}")
    plt.axis("off")
plt.show()

```



```
import torch.nn as nn
import torch.nn.functional as F
import cv2
import numpy as np
```

```
class CNNModel(nn.Module):
    def __init__(self):
        super(CNNModel, self).__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(32 * 128 * 128, 128) # 256/2 = 128 after pool
        self.fc2 = nn.Linear(128, 2)
```

```
    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = self.pool(F.relu(self.conv2(x)))
        self.feature_maps = x # Save for Grad-CAM
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = CNNModel().to(device)
model.eval()
```

```
→ CNNModel(
  (conv1): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=524288, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=2, bias=True)
)
```

```
def save_gradients_hook(module, grad_input, grad_output):
    model.feature_maps_grad = grad_output[0]
```

```
model.pool.register_full_backward_hook(save_gradients_hook)
```

```
→ <torch.utils.hooks.RemovableHandle at 0x7d6ea582fc50>
```

```
def generate_gradcam(model, input_tensor, class_idx):
    model.zero_grad()
    output = model(input_tensor)
    pred = output[:, class_idx]
    pred.backward()

    gradients = model.feature_maps_grad
    activations = model.feature_maps.detach()

    pooled_gradients = torch.mean(gradients, dim=[0, 2, 3])

    for i in range(activations.shape[1]):
        activations[:, i, :, :] *= pooled_gradients[i]

    heatmap = torch.mean(activations, dim=1).squeeze()
    heatmap = F.relu(heatmap)
    heatmap -= heatmap.min()
```

```

heatmap /= heatmap.max()
return heatmap.cpu()

image, label = dataset[0]
input_tensor = image.unsqueeze(0).to(device)
input_tensor.requires_grad = True

output = model(input_tensor)
predicted_class = output.argmax(dim=1).item()
heatmap = generate_gradcam(model, input_tensor, predicted_class)

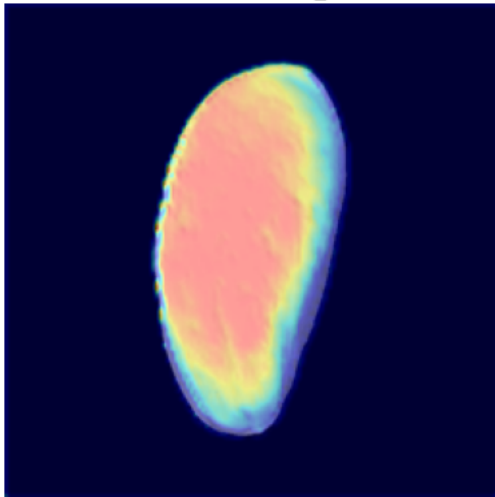
img_np = image.squeeze().numpy()
img_np = (img_np * 0.125 + 0.5) * 255
img_np = np.clip(img_np, 0, 255).astype(np.uint8)

heatmap_np = cv2.resize(heatmap.numpy(), (img_np.shape[1], img_np.shape[0]))
heatmap_color = cv2.applyColorMap(np.uint8(255 * heatmap_np), cv2.COLORMAP_JET)
img_rgb = cv2.cvtColor(img_np, cv2.COLOR_GRAY2BGR)
superimposed_img = cv2.addWeighted(heatmap_color, 0.4, img_rgb, 0.6, 0)
plt.imshow(superimposed_img[...,:-1])
plt.title(f"Grad-CAM for: {class_name[predicted_class]}")
plt.axis('off')
plt.show()

```



Grad-CAM for: Kirmizi_Pistachio



Start coding or [generate](#) with AI.